Siligent Dental AI Assistant Handoff

1. Product Summary

Siligent Dental AI Assistant is a local-first dental front-office AI application. It helps administrative users generate and review dental communications using local Ollama models, structured templates, uploaded context, and controlled runtime fields.

Current production path:

- Frontend: React + TypeScript + Vite, served through Nginx in Docker.
- Backend: FastAPI.
- Database: PostgreSQL with pgvector.
- Model runtime: Ollama running on the host machine.
- Deployment: Docker Compose for `frontend`, `api`, and `db`; Ollama is not containerized.
- Primary user roles: `admin` and `staff`.

Current product workflows:

- Insurance denial letter generation.
- Insurance verification.
- Email exchange reply drafting.
- Template library management.
- Model routing/settings.
- System health and audit review.

2. Repository Map

Important locations:

- `frontend/` - React application.
- `frontend/src/pages/` - Main UI pages.
- `frontend/src/components/` - Reusable UI components.
- `frontend/src/lib/` - Frontend API client, auth, formatting, runtime field helpers, and task state.
- `api/main.py` - FastAPI app, route registration, auth dependencies, request handlers.
- `api/schemas.py` - API request and response schemas.
- `services/` - Backend business logic, storage, model calls, prompt handling, generation, guardrails.
- `prompts/` - Version-controlled prompt files used by the app.
- `data/payer_references/` - Local payer reference files used for insurance verification.
- `docs/` - Product-specific supporting documents and field mapping specs.
- `docker-compose.yml` - Runtime composition for database, backend, and frontend.
- `start.sh` - Starts the Dockerized app stack after checking host Ollama.
- `stop.sh` - Stops the Dockerized app stack and attempts to offload Ollama models.
- `installer.sh` - Installer-style dependency, environment, port, and runtime checker.
- `.env.example` - Environment variable template.
- `requirements.txt` - Python dependencies.
- `frontend/package.json` - Frontend dependencies and build scripts.
- `tests/` - Backend test suite.

3. Runtime Architecture

The browser loads the React app from `http://localhost:3000`. The React app calls the FastAPI backend at `http://localhost:8000/api`. The backend stores structu

red application data in PostgreSQL and stores uploaded file content under `data/uploads` through the mounted `./data:/app/data` volume.

Ollama runs directly on the host at `http://localhost:11434`. Inside Docker, the API reaches Ollama through `http://host.docker.internal:11434`.

Runtime services:

- `db`: PostgreSQL 16 with pgvector, exposed on host port `5434`.
- `api`: FastAPI backend, exposed on host port `8000`.
- `frontend`: React static app served by Nginx, exposed on host port `3000`.
- `ollama`: Host process, expected on host port `11434`.

4. Environment Configuration

Start from:

```bash
cp .env.example .env
```

Important variables:

- `OLLAMA_URL`: Host Ollama URL for non-container execution. Compose overrides this to `http://host.docker.internal:11434`.
- `MODEL_NAME`: Default local model, currently `qwen3.5:4b`.
- `OLLAMA_GENERATE_TIMEOUT_SEC`: Generation timeout, currently `180`.
- `OLLAMA_NUM_PREDICT`: Max model output tokens, currently `1024`.
- `OLLAMA_THINK`: Whether to use model thinking mode, currently `false`.
- `OLLAMA_KEEP_ALIVE`: Ollama model keep-alive, currently `0` to unload after use.
- `AUTH_ENABLED`: Auth toggle, currently `true`.
- `AUTH_SESSION_HOURS`: Session duration, currently `12`.
- `ALLOW_SELF_REGISTER`: Self-registration toggle, currently `false`.
- `API_KEY`: Optional API key gate. Blank means API key check is disabled.
- `CORS_ORIGINS`: Frontend origin, currently `http://localhost:3000`.
- `DATABASE_URL`: Postgres connection string.
- `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_PORT`: Compose database settings.

For deployment beyond local development, change default database credentials before starting the stack.

5. Starting and Stopping

Prerequisites:

- Docker Desktop or Docker Engine is running.
- Ollama is installed on the host.
- The selected model is pulled locally.

Pull the default model if needed:

```bash
ollama pull qwen3.5:4b
```

Start the app:

```bash
./start.sh
```

Installer-style setup/check:

```bash
./installer.sh
./installer.sh --check-only
./installer.sh --no-start
```

Stop the app:

```bash
./stop.sh
```

Access points:

- App: `http://localhost:3000`
- API health/docs: `http://localhost:8000/docs`
- Database: `localhost:5434`
- Ollama: `http://localhost:11434`

6. Authentication and Roles

Authentication is enabled by default.

Bootstrap behavior:

1. Open `http://localhost:3000`.
2. If no users exist, the app shows the registration/bootstrap experience.
3. Create the first user with a username and password.
4. The first user becomes the admin account.
5. After bootstrap, users log in through the login screen.

Roles:

- `admin`: Can manage shared templates, template types, model preferences, users, field dictionary entries, and audit/system areas.
- `staff`: Can use generation workflows and personal templates, but cannot manage shared system-level configuration.

Operator rule:

- Create one admin owner first.
- Use admin access to create staff accounts.
- Do not share admin credentials with day-to-day staff users.

7. How to Use Each Feature

7.1 Denial Letter Generation

Purpose:

Use this flow when an insurance claim has been denied and staff need a professional appeal or denial-response letter from the Siligent provider/front-office perspective.

Who should use it:

- Staff users can generate and edit letters.
- Admin users can also manage the shared denial letter template used by the workflow.

Before starting:

- Confirm the payer name.
- Confirm the patient name.
- Confirm the date of service.
- Confirm the claim/reference number if available.
- Confirm the procedure description and procedure code.
- Confirm the denial code and denial reason.
- Prepare the appeal basis: the specific reason Siligent is asking the payer to reconsider.

Step-by-step:

1. Log in to the app.
2. Open the denial letter workflow.
3. Confirm the workflow is for denial letters; the user does not need to manually choose a separate purpose type.
4. Enter patient details exactly as they should appear in the letter.
5. Enter payer details. If the payer address is unknown, do not invent one; leave it blank or use the approved office process.
6. Enter the denial code and denial reason from the EOB or payer notice.
7. Enter the procedure description and procedure code.
8. Enter the appeal basis in plain English. Example: "The procedure was medically necessary due to documented infection and symptoms."
9. Generate the draft.
10. Review the full letter before use.
11. Edit the draft if wording needs to be adjusted.
12. Copy, download, or print/save as PDF after human review.

What good input looks like:

- Patient name is complete and spelled correctly.
- Denial code matches the payer notice.
- Appeal basis explains why the denial should be reconsidered.
- Procedure code and service date match the claim.
- Provider name and NPI are accurate if included.

What the output should contain:

- A professional letter format.
- A subject or reference line.
- Patient, payer, claim, procedure, and denial context.
- A clear appeal rationale tied to the denial reason.
- A provider/front-office closing.

Review checklist:

- No invented patient names, dates, providers, phone numbers, payer addresses, claim IDs, or procedure codes.
- Denial reason and appeal basis are logically connected.
- The tone is professional and administrative, not casual or AI-narrated.
- The letter is ready for a human staff member or provider to approve.

Common issues:

- If the letter says `Not provided`, add the missing field and regenerate.
- If the appeal rationale is generic, improve the appeal basis field and regenerate.
- If the letter includes facts not entered by the user or not present in the source context, remove them before use.

7.2 Insurance Verification

Purpose:

Use this flow to summarize locally stored payer reference information and determine whether a requested condition/procedure appears covered, not covered, or needs manual review based on available local policy text.

Who should use it:

- Staff users can run verifications.
- Admin users maintain payer references and shared templates outside the normal staff workflow.

Before starting:

- Confirm the payer.
- Confirm member ID and group number if available.
- Confirm patient date of birth.
- Confirm plan type if known.
- Enter the requested condition and requested procedure when asking whether a specific procedure is covered.

Step-by-step:

1. Log in to the app.
2. Open the insurance verification workflow.
3. Select the payer.
4. Enter patient/member details.
5. Enter the requested condition. Example: "symptomatic irreversible pulpitis" or "tooth infection."
6. Enter the requested procedure. Example: "root canal therapy" or "D3310 anterior RCT."
7. Generate the verification.
8. Review the structured output.
9. Read the coverage verdict and rationale.
10. Use the result as a staff aid, then confirm with payer systems or office policy if the result is high-impact.

What good input looks like:

- Payer selection matches a file under `data/payer_references/`.
- Requested procedure is specific enough to compare against policy language.
- Member and plan details are entered without placeholders.
- Unknown fields are left blank rather than filled with guessed values.

What the output should contain:

- Payer and patient/member context.
- Coverage findings.
- Coverage verdict for the requested condition/procedure.
- Rationale tied to payer reference language.
- Limitations or exclusions when available.
- Next actions for staff.

How to interpret the verdict:

- `Covered`: The local payer reference supports coverage for the requested procedure/context.
- `Not covered`: The local payer reference indicates exclusion or no coverage.
- `Manual review needed`: The local payer reference is incomplete, ambiguous, or does not clearly answer the specific question.

Review checklist:

- The verdict is tied to local payer reference data.
- The result does not claim real-time payer eligibility.
- The output does not invent plan benefits absent from the payer reference.
- The user understands this is not a live payer transaction.

Common issues:

- If the payer is missing, add or correct the payer reference file.
- If the verdict is too cautious, improve local payer reference content.
- If requested procedure is vague, regenerate with a more specific procedure and code.

7.3 Email Exchange Drafting

Purpose:

Use this flow to draft a personalized reply to a pasted or uploaded email thread. The app does not connect directly to a mailbox; the user provides the thread context manually.

Who should use it:

- Staff users can generate and edit replies.
- Admin users can manage templates and model settings that influence generated replies.

Before starting:

- Copy the relevant email thread text.
- Identify who the reply is going to.
- Identify the sender/office role.
- Identify the topic.
- Decide the next step the email should ask for or communicate.
- Remove unnecessary unrelated thread content if it may confuse the model.

Step-by-step:

1. Log in to the app.
2. Open the email exchange workflow.
3. Paste the email thread text or upload relevant context.
4. Enter recipient name or role if known.
5. Enter sender or office role if needed.
6. Enter the topic of the reply.
7. Enter the intended next step. Example: "Ask the patient to call the office to update insurance."
8. Generate the draft.
9. Review the response for accuracy and tone.
10. Edit the draft directly before sending it in the actual email system.
11. Copy the final approved text into the mailbox or external system.

What good input looks like:

- Thread text contains the message being answered.
- Runtime fields explain the intended action.
- The next step is explicit.
- The user does not rely on the model to infer dates, phone numbers, insurance details, or promises not present in the thread.

What the output should contain:

- A professional response from the Siligent provider/front-office perspective.
- A clear answer to the received email.
- A specific next step.
- No unresolved placeholders.
- No `Not provided` filler language.

Review checklist:

- The response answers the actual latest email in the thread.
- The tone is appropriate for dental front-office communication.
- No patient facts, dates, phone numbers, or insurance details were invented.
- The draft is edited and approved before being sent externally.

Common issues:

- If the response says `Not provided`, add missing runtime details and regenerat
e.
- If the draft answers the wrong part of the thread, paste only the relevant lat
est exchange and regenerate.
- If the reply is too generic, provide a more specific next step and topic.

7.4 Template Library

Purpose:

Use this area to manage reusable wording for letters, emails, and workflow-speci
fic templates.

Who should use it:

- Admin users manage shared canonical templates.
- Staff users can use templates and save personal variants where permitted.

Step-by-step for reviewing a template:

1. Log in to the app.
2. Open Template Library.
3. Browse by template type, tags, or visible template cards.
4. Open a template in the workspace.
5. Review the wording, variables, and intended use case.

Step-by-step for editing a shared template as admin:

1. Open the shared template.
2. Edit the plain-language wording.
3. Confirm variables still represent fields that can be filled at runtime.
4. Avoid replacing placeholders with one-off patient values.
5. Save the template.
6. Run a generation test for that template type.

Step-by-step for staff personal templates:

1. Open an existing template.
2. Make a personal wording adjustment.
3. Save as a personal variant if the UI permits.
4. Use the personal variant for future drafts.

Variable guidance:

- Templates should contain placeholders/variables, not real patient values.
- Variables should represent information that will be supplied later.

- Example concept: use patient-name variable behavior rather than typing a real patient name into a reusable template.

Review checklist:

- Template wording is reusable.
- Variables are clear to a non-technical user.
- The template does not contain real PHI.
- The purpose type matches the workflow that will use it.

7.5 Template Draft Generation

Purpose:

Use this when an admin wants the model to draft a starter template that can then be edited and saved by a human.

Step-by-step:

1. Open the template creation or template library workflow.
2. Choose the template type or create the needed type if allowed.
3. Provide the intended purpose and required fields.
4. Generate a template draft.
5. Review the draft carefully.
6. Replace any specific example values with reusable variables/placeholders.
7. Edit wording until it fits the office's preferred tone.
8. Save as the canonical shared template if it is ready for staff use.
9. Run one generation test using realistic runtime fields.

Review checklist:

- The generated template uses placeholders rather than fake patient values.
- The template is reusable across many patients or claims.
- The template does not include unsupported legal, clinical, or coverage claims.

7.6 Model Settings

Purpose:

Use this area to control which locally installed Ollama model is used globally or for specific use cases.

Who should use it:

- Admin users only.

Step-by-step:

1. Confirm the desired model is installed locally with `ollama list`.
2. Log in as admin.
3. Open Model Settings.
4. Review available local models.
5. Choose a global model if one model should be used for all workflows.
6. Choose per-use-case models if different workflows need different models.
7. Save settings.
8. Run a small generation test for each affected workflow.

Rules:

- The app does not pull models automatically.
- Model names must match local Ollama model names.
- If a model fails during generation, check Ollama logs and memory availability.

7.7 System Health and Audit

Purpose:

Use this area to confirm the app is operating and to review administrative audit events.

Who should use it:

- Admin users.

Step-by-step:

1. Log in as admin.
2. Open System Health.
3. Confirm the API is reachable.
4. Confirm model configuration is visible.
5. Review audit events for account, template, and generation-related activity.
6. If health checks fail, verify Docker, API, database, and Ollama are running.

Troubleshooting order:

1. Run `docker compose ps`.
2. Check API logs with `docker compose logs -f api`.
3. Check frontend availability at `http://localhost:3000`.
4. Check API availability at `http://localhost:8000/`.
5. Check Ollama with `ollama list` and `ollama ps`.
6. Restart with `./stop.sh` then `./start.sh` if needed.

8. Prompt and Generation System

Prompt files live under `prompts/`.

Prompt routing is centralized in:

- `services/prompt_registry.py`

Generation-related services:

- `services/generation.py` - denial letters, email drafts, insurance verification, template drafts.
- `services/prompt_engine.py` - prompt loading/rendering helpers.
- `services/template_runtime.py` - runtime field normalization and template rendering.
- `services/document_pipeline.py` - multi-document extraction and smart composer support.
- `services/email_thread.py` - thread analysis and reply generation.
- `services/email_guardrails.py` - email draft validation.
- `services/autonomy_policy.py` - factual grounding controls.
- `services/ollama_client.py` - Ollama health/model/generation API client.

Operational rule:

- Prompt text should remain in `prompts/**/*.txt`.
- Runtime code should select prompts through the prompt registry.
- Do not scatter prompt strings directly across UI or route handlers.

9. Data and Persistence

Primary persistence:

- PostgreSQL through `DATABASE_URL`.

Postgres-backed stores include:

- Users and sessions.
- Templates.
- Template types.
- Field dictionary.
- Model preferences.
- Upload metadata.
- Audit events.

Filesystem data:

- `data/payer_references/` - payer coverage reference text files.
- `data/uploads/` - uploaded source files and extracted document content.
- `prompts/` - read-only prompt templates mounted into the API container.

10. API Surface

Primary endpoints:

- `GET /`
- `GET /api/auth/bootstrap`
- `POST /api/auth/register`
- `POST /api/auth/login`
- `POST /api/auth/logout`
- `GET /api/auth/me`
- `GET /api/health`
- `GET /api/models`
- `GET /api/model-preferences`
- `PUT /api/model-preferences`
- `GET /api/template-types`
- `POST /api/template-types`
- `GET /api/field-dictionary`
- `PUT /api/field-dictionary/{field_key}`
- `DELETE /api/field-dictionary/{field_key}`
- `GET /api/denial-codes`
- `GET /api/payers`
- `POST /api/denial-letters/generate`
- `POST /api/emails/generate`
- `POST /api/email-thread/generate`
- `POST /api/insurance-verification/generate`
- `POST /api/dentrix/resolve-template-fields`
- `POST /api/document-pipeline/generate`
- `POST /api/template-drafts/generate`
- `GET /api/templates`
- `POST /api/templates`
- `DELETE /api/templates/{index}`
- `GET /api/audit-events`

API errors are returned as structured JSON with an error code and human-readable message.

11. Developer Code Map

Use this section when handing the product to an engineer who needs to understand where each workflow is implemented.

Backend Entry Points

- `api/main.py`

  - `create_app` creates the FastAPI app.
  - `register_error_handlers` maps exceptions into structured JSON errors.
  - `register_routes` defines all API routes.
  - `require_api_key`, `require_current_user`, and `require_admin` enforce acces
s control.
  - `_resolve_model_for_use_case` chooses the Ollama model for a workflow.
  - `_parse_runtime_fields`, `_prepare_runtime_values`, and `_merge_extracted_ru
ntime_values` prepare user/runtime fields before generation.
- `api/schemas.py`
  - Contains request/response models for auth, templates, denial letters, insura
nce verification, email threads, document pipeline, audit events, and Dentrix ma
pping.

Denial Letter Workflow

- Frontend:
  - `frontend/src/pages/SmartComposerPage.tsx` handles denial-letter form state,
 runtime fields, draft display, editing, and actions.
- Backend route:
  - `POST /api/denial-letters/generate` in `api/main.py`.
- Backend services:
  - `services/generation.py`
  - `generate_denial_letter` renders the denial prompt and calls Ollama.
  - `services/prompt_registry.py`
  - `resolve_denial_prompt` maps denial code to prompt path.
  - `services/constants.py`
  - Stores supported denial codes and descriptions.
  - `services/autonomy_policy.py`
  - `enforce_draft_grounding_or_raise` checks generated drafts for unsupported h
igh-risk facts.
- Prompts:
  - `prompts/denial_letters/*.txt`

Insurance Verification Workflow

- Frontend:
  - `frontend/src/pages/InsuranceVerificationPage.tsx` handles payer selection,
patient/member fields, requested condition/procedure fields, and verification di
splay.
- Backend route:
  - `POST /api/insurance-verification/generate` in `api/main.py`.
- Backend services:
  - `services/generation.py`
  - `list_available_payers` lists payer reference files.
  - `payer_to_filename` and `filename_to_payer_display` convert between UI payer
 names and filenames.
  - `_resolve_payer_reference_path` prevents payer-reference path traversal.
  - `load_payer_reference` loads selected payer reference text.
  - `generate_insurance_verification` builds the prompt, calls Ollama, normalize
s JSON, and returns structured output.
  - `services/verification.py`
  - `extract_json_object` extracts model JSON.
  - `normalize_verification_summary` normalizes verification fields.
  - `enforce_grounded_verification_summary` enforces payer-reference grounding.
  - `summary_to_text` converts structured summary to display text.
- Prompt/data:
  - `prompts/insurance_verification.txt`
  - `data/payer_references/*.txt`

Email Exchange Workflow

- Frontend:

  - `frontend/src/pages/SmartComposerPage.tsx` handles email-thread mode, pasted /uploaded context, runtime fields, generation state, editable output, and output actions.
- Backend route:
  - `POST /api/email-thread/generate` in `api/main.py`.
- Backend services:
  - `services/email_thread.py`
  - `normalize_email_thread` trims and normalizes input thread text.
  - `extract_latest_message` isolates the latest message.
  - `build_email_thread_context` builds thread context.
  - `heuristic_analyze_email_thread` provides fallback analysis.
  - `analyze_email_thread_with_model` analyzes intent, urgency, tone, and entities.
  - `recommend_email_templates` recommends matching templates.
  - `generate_email_thread_reply` generates the final reply.
  - `ensure_email_reply_has_no_missing_markers` rejects unresolved placeholders or `Not provided` output.
  - `services/email_guardrails.py`
  - `build_enforced_email_prompt`, `generate_with_guardrails`, and `is_role_and_purpose_compliant` enforce role/purpose alignment.
- Prompts:
  - `prompts/email_thread/analyze_thread.txt`
  - `prompts/email_thread/generate_reply.txt`
  - `prompts/emails/*.txt`

Template Library and Template Drafting

- Frontend:
  - `frontend/src/pages/TemplateLibraryPage.tsx` handles browsing, filtering, editing, variables, tags, shared/personal saves, and template draft creation.
  - `frontend/src/components/RuntimeFieldsEditor.tsx` renders runtime field inputs.
  - `frontend/src/components/TemplateSaveBar.tsx` handles template save UI.
  - `frontend/src/lib/templateVariables.ts` converts natural-language labels and canonical placeholders.
  - `frontend/src/lib/templateRuntime.ts` extracts/renders placeholders on the client.
  - `frontend/src/lib/templateTags.ts` parses tag input.
  - `frontend/src/lib/templateSave.ts` validates template save input.
- Backend routes:
  - `GET /api/templates`
  - `POST /api/templates`
  - `DELETE /api/templates/{index}`
  - `GET /api/template-types`
  - `POST /api/template-types`
  - `POST /api/template-drafts/generate`
- Backend services:
  - `services/template_store.py` and `services/postgres_store.py` store templates.
  - `services/template_type_store.py` normalizes and manages template types.
  - `services/default_templates.py` seeds missing shared default templates.
  - `services/template_runtime.py`
  - `extract_template_placeholders` finds required placeholders.
  - `normalize_runtime_fields` normalizes runtime values.
  - `expand_runtime_field_aliases` maps aliases to canonical runtime keys.
  - `render_template_with_runtime_fields` renders saved templates.
  - `runtime_fields_to_json_context` sends runtime fields to the model as JSON context.
  - `services/generation.py`
  - `generate_template_draft` creates model-assisted reusable template drafts.
  - `_repair_template_to_placeholder_only` pushes generated templates toward placeholders instead of fake values.

Document Upload and Smart Composer Internals

- Frontend:
  - `frontend/src/components/SelectedFilesList.tsx` displays selected uploads.
  - `frontend/src/lib/uploadConfig.ts` defines upload limit and accepted file types.
  - `frontend/src/lib/generationTasks.tsx` preserves generation task state while navigating.
- Backend route:
  - `POST /api/document-pipeline/generate` in `api/main.py`.
- Backend services:
  - `services/document_pipeline.py`
  - `validate_upload_constraints` enforces upload count.
  - `extract_text_from_payload` extracts text from supported uploads.
  - `_extract_text_from_pdf`, `_extract_text_from_docx`, `_extract_text_from_rtf`, and `_decode_text_payload` handle file formats.
  - `extract_document_content` prepares extracted content.
  - `build_document_context` and `build_document_generation_context` prepare prompt context.
  - `heuristic_detect_template_type` and `detect_template_type_with_model` classify document purpose.
  - `recommend_templates` scores template matches.
  - `generate_structured_output` produces structured draft output.
  - `stabilize_structured_output` repairs weak drafts.
  - `services/upload_store.py` and `services/postgres_store.py` store upload metadata/content.

Model, Settings, and Ollama

- Frontend:
  - `frontend/src/pages/ModelSettingsPage.tsx` handles model preferences, account creation, and admin settings.
- Backend routes:
  - `GET /api/models`
  - `GET /api/model-preferences`
  - `PUT /api/model-preferences`
- Backend services:
  - `services/ollama_client.py`
  - `OllamaClient` wraps model listing, health checks, and generation calls.
  - `services/model_preferences_store.py` and `services/postgres_store.py` persist model routing.
  - `services/config.py`
  - `Settings` stores runtime config.
  - `load_settings` reads environment values.

Authentication, Users, and Permissions

- Frontend:
  - `frontend/src/pages/LoginPage.tsx` handles login/bootstrap UI.
  - `frontend/src/lib/auth.tsx` stores auth state and session behavior.
  - `frontend/src/lib/permissions.ts` defines frontend admin/staff permission helpers.
  - `frontend/src/lib/api.ts` attaches auth token/API key headers and exposes API methods.
- Backend:
  - `api/main.py`
  - `require_current_user` and `require_admin` protect routes.
  - `_resolve_registration_role` enforces bootstrap/admin/staff registration rules.
  - `services/auth_store.py` handles file-store auth fallback.
  - `services/postgres_store.py` contains `PostgresAuthStore` for DB-backed auth

.

Audit, Health, and Output Actions

- Frontend:
  - `frontend/src/pages/SystemHealthPage.tsx` displays health and audit information.
  - `frontend/src/components/Notice.tsx` displays success/error messages.
  - `frontend/src/components/OutputActions.tsx` supports copy, text download, rich email output, and print/save PDF behavior.
- Backend:
  - `GET /api/health` in `api/main.py`.
  - `GET /api/audit-events` in `api/main.py`.
  - `services/audit_store.py` handles file-store audit fallback.
  - `services/postgres_store.py` contains `PostgresAuditStore` for DB-backed audit events.

Dentrix Field Mapping

- Backend route:
  - `POST /api/dentrix/resolve-template-fields` in `api/main.py`.
- Backend service:
  - `services/dentrix_mapping.py`
  - `resolve_dentrix_template_fields` maps Dentrix-style claim data to template fields.
  - Helper functions load mapping specs, traverse nested records, filter rows by claim, and return resolved/missing fields.
- Mapping spec:
  - `docs/DENTRIX_FIELD_MAPPING_SPEC.json`

Test Map

- `tests/test_api.py` covers API behavior.
- `tests/test_auth_store.py` covers auth store behavior.
- `tests/test_generation_security.py` covers generation security/path behavior.
- `tests/test_verification.py` covers insurance verification normalization/grounding.
- `tests/test_email_thread.py` and `tests/test_email_guardrails.py` cover email-thread behavior.
- `tests/test_document_pipeline.py` covers upload/document pipeline behavior.
- `tests/test_template_runtime.py` and `tests/test_template_store.py` cover templates/runtime rendering.
- `tests/test_dentrix_mapping.py` covers Dentrix mapping.
- `tests/test_ollama_client.py` covers Ollama client behavior.
- `tests/test_prompt_engine.py` covers prompt rendering.

12. Verification Checklist

Run backend tests:

```bash
python3 -m unittest discover -s tests -p 'test_*.py' -v
```

Run frontend build:

```bash
cd frontend
npm run build
```

Check Docker stack:

```bash
docker compose ps
```

Check API:

```bash
curl -sS http://localhost:8000/
```

Check Ollama:

```bash
ollama ps
ollama list
```

Manual smoke test:

1. Open `http://localhost:3000`.
2. Bootstrap/login as admin.
3. Confirm System Health loads.
4. Generate one denial letter.
5. Generate one insurance verification.
6. Generate one email exchange reply.
7. Edit a generated draft.
8. Save a personal template.
9. As admin, save/update a shared template.
10. Stop the app with `./stop.sh` and confirm models are offloaded with `ollama ps`.

13. Known Operational Constraints

- Ollama must be running on the host before generation works.
- The selected model must already be pulled locally.
- Vision-capable model behavior depends on the local model installed in Ollama.
- Insurance verification depends on local payer reference files; unsupported pay
er data cannot be inferred reliably.
- Generated content must be reviewed by a human user before being sent, printed,
 or filed.
- The app is intended for local/private deployment, not public internet exposure
 without additional infrastructure hardening.

14. Maintenance Notes

Adding a payer:

1. Add a text file under `data/payer_references/`.
2. Use lowercase filename formatting where spaces become underscores, for exampl
e `delta_dental.txt`.
3. Restart or refresh the app if the payer list does not immediately appear.

Adding or changing a prompt:

1. Edit the relevant file under `prompts/`.
2. Confirm the prompt is registered in `services/prompt_registry.py`.
3. Run backend tests.
4. Run a manual generation test for the affected workflow.

Adding a template type:

1. Admin creates the new type through the UI.
2. Add or update the shared canonical template for that type.
3. Confirm required variables are understandable to staff users.

Changing model routing:

1. Pull the model locally with Ollama.
2. Confirm it appears in the Model Settings page.
3. Set global or per-use-case preference.
4. Run a generation smoke test.

15. Handoff Acceptance Criteria

A new owner should be able to accept the project when they can:

- Start the app with `./start.sh` or validate setup with `./installer.sh`.
- Log in or bootstrap the first admin account.
- Create a staff account.
- Generate and edit a denial letter.
- Generate and interpret insurance verification output.
- Generate and edit an email exchange reply.
- Save and update templates according to role permissions.
- Change model settings to an installed local Ollama model.
- Run backend tests and frontend build successfully.
- Stop the stack cleanly with `./stop.sh`.